**NATURAL LANGUAGE PROCESSING (NLP)**

**UNIT IV**

---

## SEMANTIC PARSING II

---

1. **Predicate-Argument Structure,**
2. **Meaning Representation Systems**

### 1. PREDICATE-ARGUMENT STRUCTURE

**Topics:**

1. **Resources**
2. **Systems**
3. **Software**

**Q1: Explain Predicate-Argument Structure. (Resources, Systems, Software)**

**OR**

**Write short notes on Predicate-Argument Structure found in NLP.**

**Q2: What are the Syntactic and Semantic Constraints on Predicate-Argument Structure?**

## Introduction

**Predicate-Argument Structure (PAS**), also known as **Semantic Role Labeling (SRL)**, is the process of identifying the semantic roles of various arguments associated with a predicate (such as a verb, noun, or adjective) in a sentence. The goal is to determine **"who did what to whom, when, where, and how."** For a given predicate, the system identifies all constituents in the sentence that act as arguments and assigns them specific semantic labels.

The knowledge of PAS has significant real-world applications, particularly in the area of **Information Extraction**. By understanding the semantic relationships in a text, systems can extract structured information from unstructured sources, enabling more advanced question answering, text summarization, and machine translation.

## RESOURCES

In Natural Language Processing, the analysis of Predicate-Argument Structure has transitioned from rule-based methods to data-oriented approaches. This shift was enabled by the development of large, semantically annotated corpora. These resources provide the foundation for training and evaluating machine learning models for **Semantic Role Labeling** (SRL). The two most influential resources are **FrameNet** and **PropBank.**

### 1. FrameNet

FrameNet is a resource based on the theory of **frame semantics**. This theory suggests that the meaning of a word (a predicate) is understood by evoking a conceptual structure or scenario, known as a **semantic frame**.

- **Core Concepts:**
  - **Semantic Frame:** Represents a specific situation or event. For example, the AWARENESS frame describes a scenario where a conscious entity (Cognizer) has a certain piece of information (Content).
  - **Frame Elements (FEs):** These are semantic roles specific to a frame. They are the participants and props in the scenario described by the frame (e.g., Cognizer, Content, Topic.
  - **Lexical Unit (LU):** This is the pairing of a predicate (a word) with the specific frame it evokes. A single word can be part of multiple LUs if it has multiple meanings (is polysemous). For example, verb *break* can evoke:

- The **COMPLIANCE** frames when it means "to fail to observe an agreement."
- The **CAUSE_TO_FRAGMENT** frame means "to cause to separate into pieces."

- **Annotation Process:**

1. A semantic frame is defined.
2. A set of frame-specific roles (Frame Elements) is created for that frame.
3. Predicates that can evoke this frame are identified.
4. Sentences containing these predicates are annotated by identifying the arguments and labeling them with the corresponding Frame Elements.

- **Example: The AWARENESS Frame**

The below diagram illustrates the **AWARENESS frame**, its associated Frame Elements, and a sample of predicates (verbs and nouns) that can evoke it.

**Diagram: FrameNet example**

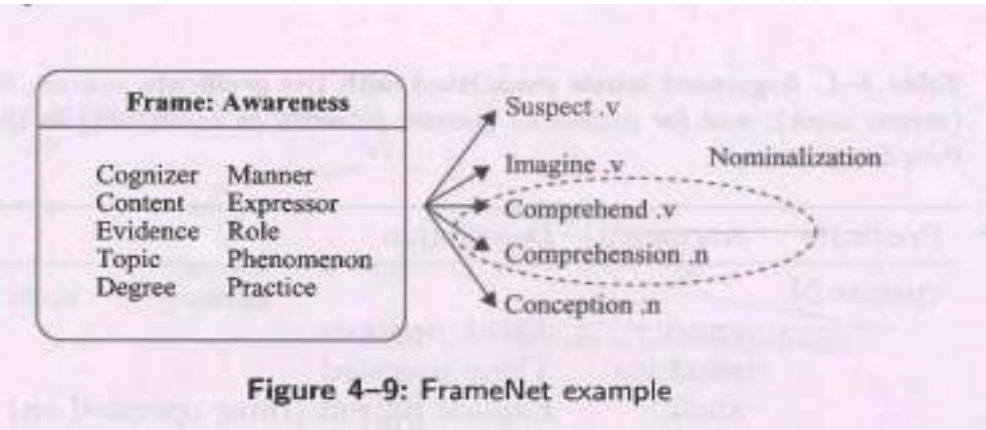



Figure 4–9: FrameNet example

The following sentences show this frame in action:

1. **[Cognizer We] [Predicate:verb believe] [Content it is a fair and generous price]**

2. **No doubt existed as to [Cognizer our] [Predicate: noun comprehension] [Content of it]**

**FrameNet** contains annotations for a wide variety of predicates, including verbs, nouns, adjectives, and prepositions, using sentences from the British National Corpus (BNC).

---

### *2. PropBank (Proposition Bank)*

PropBank offers a different, more "linguistically neutral" approach. It is built upon the syntactic structures of the Penn Treebank and primarily focuses on annotating the arguments of verbs.

- **Core Concepts:**
  - PropBank restricts argument boundaries to the exact syntactic constituents found in the Penn Treebank parse trees.
  - It defines two types of arguments: core and adjunctive.

- **<u>Argument Types</u>:**
  - ✓ **Core Arguments (ARGN):** These are arguments whose semantic role is dependent on the specific predicate. They are labeled numerically from ARG0 to ARG5. While there are general tendencies (e.g., ARG0 is often the PROTO-AGENT (usually the subject of a transitive verb, ARG1 is the PROTO-PATIENT (usually its direct object of the transitive verb)), their precise meaning is defined for each predicate in a corresponding **frames file**.

- ✓ **Adjunctive Arguments (ARGM-X):** These are modifier arguments whose meaning is consistent across all predicates. They represent general notions like temporal (ARGM-TMP), locative (ARGM-LOC), manner (ARGM-MNR), etc.

- **The table below** shows how the meaning of core arguments changes with the predicate. For operate.01, ARG1 is the "Thing operated," while for author.01, ARG1 is the "Text authored."

**Table : Argument labels for operate.01 and author.01**

**Table 4–1: Argument labels associated with the predicate *operate.01* (sense: work), and for *author.01* (sense: to write or construct) in the PropBank corpus**

| Predicate | Argument | Description |
|---|---|---|
| operate.01 | | |
| | ARG0 | Agent, operator |
| | ARG1 | Thing operated |
| | ARG2 | Explicit patient (thing operated on) |
| | ARG3 | Explicit argument |
| | ARG4 | Explicit instrument |
| author.01 | | |
| | ARG0 | Author, agent |
| | ARG1 | Text authored |

**The Table below** lists some common adjunctive arguments, which maintain their meaning regardless of the predicate.

**Table : List of adjunctive arguments in PropBank—ARGMS**

Table 4–2: List of adjunctive arguments in PropBank—ARGMs

| Tag | Description | Examples |
|---|---|---|
| ARGM-LOC | Locative | *the museum, in Westborough, Mass.* |
| ARGM-TMP | Temporal | *now, by next summer* |
| ARGM-MNR | Manner | *heavily, clearly, at a rapid rate* |
| ARGM-DIR | Direction | *to market, to Bangkok* |
| ARGM-CAU | Cause | *In response to the ruling* |
| ARGM-DIS | Discourse | *for example, in part, Similarly* |
| ARGM-EXT | Extent | *at $38.375, 50 points* |
| ARGM-PRP | Purpose | *to pay for the plant* |
| ARGM-NEG | Negation | *not, n't* |
| ARGM-MOD | Modal | *can, might, should, will* |
| ARGM-REC | Reciprocals | *each other* |
| ARGM-PRD | Secondary Predication | *to become a teacher* |
| ARGM | Bare ARGM | *with a police escort* |
| ARGM-ADV | Adverbials | *(none of the above)* |

The example below is extracted from the ProbBank corpus along with its syntax tree representation and argument labels shown in the diagram below:

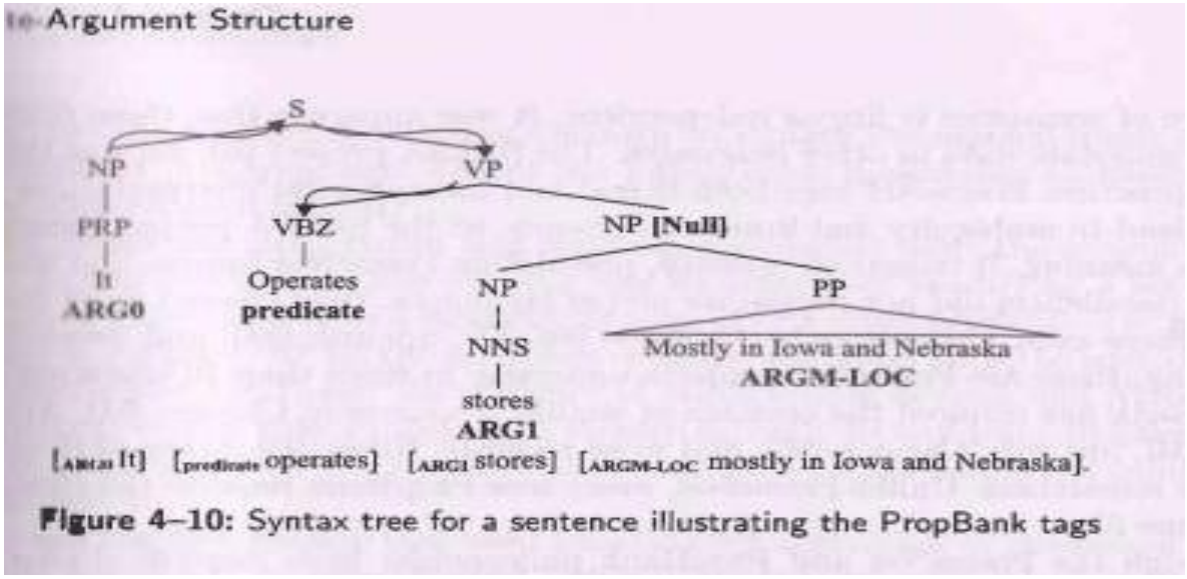



**Figure 4–10:** Syntax tree for a sentence illustrating the PropBank tags

***Other Resources:***

The methodologies of FrameNet and PropBank have inspired the creation of resources and have been adapted for numerous languages.

- **NomBank:** Inspired by PropBank, NomBank focuses on identifying and tagging the arguments of nominal predicates (nouns).
- **VerbNet:** This resource provides a richer representation by linking PropBank frames with predicate-independent thematic roles and Levin verb classes.
- **Cross-lingual Adaptation:**
  - **FrameNet** has been adapted for languages like Japanese, Spanish, and Swedish, as its frame-semantic basis is largely independent of a specific language's grammar.
  - **PropBank** has inspired similar corpora in Chinese, Arabic, Korean, Spanish, and Hindi. Unlike FrameNet, creating a new PropBank requires a new set of frame files for each language.

Thus, the success of these projects has inspired others, such as **NomBank** for noun predicates and **VerbNet**, which links PropBank frames to more general thematic roles. These philosophies have also been extended to many other languages, including Chinese, Japanese, Arabic, and Spanish, demonstrating the cross-lingual applicability of predicate-argument analysis.

**SYSTEMS**

**Introduction to Semantic Role Labeling (SRL) Systems**

The introduction of large, semantically annotated corpora like FrameNet and PropBank in the late 1990s marked a major shift in approaching predicate-argument structure recognition. Research moved from traditional, rule-based heuristic systems to more robust, data-driven machine learning frameworks. This modern approach, popularly known as **Semantic Role Labeling (SRL)**, treats the problem as a supervised classification task.

The seminal work by Gildea and Jurafsky was the first to formalize SRL in this manner. They proposed that the arguments of a predicate could be identified and labeled by mapping them to nodes in a sentence's syntactic parse tree. This formulation led to the establishment of three standard evaluation tasks that have become central to the field:

1. **Argument Identification:** The task of identifying which constituents (nodes) in a parse tree represent a valid semantic argument for a given predicate. This is a binary classification task for each node (argument vs. not an argument).

2. **Argument Classification:** Given the correct (gold-standard) argument constituents, the task is to assign the correct semantic role label (e.g., ARG0, ARG1, ARGM-LOC) to each one.

3. **Argument Identification and Classification:** A combination of the first two tasks, where the system must both identify the argument boundaries and assign the correct label. This is the complete and most challenging SRL task.

The **pseudocode** for a generic Semantic Role Labeling (SRL) algorithm is shown in the below Algorithm:

**The Semantic Role Lableing (SRL) Algorithm:**

**Procedure: SRL(Sentence) returns best Semantic Role**

**Labeling Input: Syntactic Parse of the Sentence**

1. *generate* a full syntactic parse of the sentence.
2. *identify* all the predicates
3. ***for all*** predicate Є sentence do
4. *extract* a set of features for each node in the tree relative to the predicate.
5. *classify* each feature vector using the model created in training.
6. *select* the class of highest scoring classifier
7. **return** best semantic role labelling
8. **end for**

**for each predicate in a sentence:**

1. Consider every node in the syntactic parse tree as a potential argument.
2. Extract a feature vector for each node relative to the predicate.
3. Use a trained classification model to predict a label for each node (including a "NULL" label for non-arguments).
4. Select the best-scoring label for each node to produce the final semantic role annotation.

## Sentence Representations

The following are various types of sentence representations:

### 1. Phrase Structure Grammar (PSG)

Since PropBank annotations are layered directly onto the phrase structure trees of the Penn Treebank, using a **Phrase Structure Grammar (PSG)** parse is the most natural and common approach. frameNet marks word spans in sentences to represent arguments, whereas propBank tags nodes in treebank tree with arguments. Because several high-quality statistical parsers existed that could produce phrase structure trees, and because the phrase structure representation is amenable to tagging. Gildea and Jurafsky used it. Early and influential systems extracted a rich set of features from these trees.

**Key Features from PSG:**

- **Path:** The syntactic path from the argument constituent to the predicate through the parse tree. It is a highly informative feature, represented as a string of node labels with arrows indicating upward (↑) or downward (↓) traversal (e.g., NP↑S↓VP↓VBZ).
- **Predicate:** The lemma of the predicate verb itself is a crucial feature, as argument structures are predicate-dependent.
- **Phrase Type:** The syntactic category of the constituent being classified (e.g., NP, PP, S, etc.).
- **Position:** A binary feature indicating whether the constituent appears before or after the predicate.
- **Voice:** A binary feature indicating whether the predicate is in the active or passive voice, which is critical as it often affects the position of the Agent and Patient roles.
- **Head Word:** The syntactic head word of the phrase. For example, in the phrase "the big red car," the head word is "car."

- **Subcategorization:** The phrase structure rule that expands the predicate's parent node in the parse tree (e.g., VP → VBZ NP). This captures the local syntactic frame of the predicate.

- **Verb Clustering:** To handle data sparsity and unseen verbs, predicates are grouped into semantic classes based on their co-occurrence with direct objects. This allows the model to generalize across verbs with similar meanings (e.g., *eat*, *devour*, *savor*).

- **Named Entities (NE):** Identifying if a constituent contains a named entity (e.g., PERSON, LOCATION, TEMPORAL). This is particularly useful for identifying adjunctive arguments like ARGM-LOC and ARGM-TMP.

- **Verb Sense Information:** The specific frameset ID of a predicate in PropBank (e.g., talk.01 vs. talk.02). Disambiguating the verb sense is critical because different senses have different argument structures.

- **Path Generalizations:** Due to the data-sparse nature of the full path feature, various generalization techniques are used, such as replacing non-clause nodes with a wildcard, decomposing the path into n-grams, or using only the partial path to the lowest common ancestor.

### 2. Combinatory Categorial Grammar (CCG)

While PSG paths are informative, they can be long and sparse, making them difficult to generalize. A **Combinatory Categorial Grammar (CCG)** offers a lexicalized representation that can produce shorter, more direct dependency paths between a predicate and its arguments. Features from CCG are often used to augment a primary PSG-based system.

**Key Features from CCG:**

- **Phrase Type:** In CCG, this is the category of the maximal projection between the predicate and the dependent word.
- **Categorial Path:** A concise path feature formed by concatenating three values:

(i) the category of dependent words,

(ii) the direction of dependence, and

(iii) the argument slot filled by the dependent. For example, the path between *denied* and *plans* might be (S[dcl]\NP)/NP.2 → .

- **Tree Path:** The CCG analogue of the PSG path feature, tracing the path between the predicate and the dependent through the binary CCG parse tree.

### 3. Tree-Adjoining Grammar (TAG)

A **Tree-Adjoining Grammar (TAG)** is another formalism used in SRL, primarily for its ability to effectively model long-distance dependencies in text. Systems using TAG extract features from its unique structural representations.

**Key Features from TAG:**

- **Supertag Path:** A path feature derived from a TAG structure, analogous to the PSG path.
- **Supertag:** The elementary tree frame associated with either the predicate or the argument, which provides rich lexical and structural information.
- **Surface and Deep Syntactic Roles:** TAG analysis can provide both a surface role and a deep syntactic role for an argument (e.g., *subject*, *direct object*), which helps normalize across syntactic variations like passivization.
- **Surface and Deep Subcategorization:** These features capture the argument frame of a predicate at both the surface level and a more

abstract, deep-structure level (e.g., NP0_NP1 for a transitive verb).

- **Semantic Subcategorization:** An extension of the subcategorization frame that includes semantic role information, providing a tighter link between syntax and semantics.

## SOFTWARE

Following is a list of software packages available for semantic role labeling.

- **ASSERT (Automatic Statistical Semantic Role Tagger)**

[http:://www/cemantix.org/assert.html]

A semantic role labeler trained on the English PropBank data.

- **C-ASSERT** [http://hlt030.cse.ust.hk/research/Pc-asser/]

An extension of ASSERT for the Chinese Language.

- **SwiRL** [http://www/surdeanu.name/mihal/sir/]

Another semantic role labeler trained on PropBank data.

- **Shalmaneser** ( A Shallow Semantic Parser)

[http://www/coli.uni-saarland.de/projects/salsa/shal/]

A toolchain for shallow semantic parsing based on the FrameNet data.

*****

## 2. MEANING REPRESENTATION

**Topics** :

1. Resources
2. Systems
3. Software

**Write short notes on Meaning Representation.**

**Introduction**

Meaning Representation is a deep level of semantic interpretation in Natural Language Processing whose primary objective is to transform a natural language input (like a sentence or query) into a formal, unambiguous, and canonical representation that a machine can directly act upon or execute. This process is often referred to as **deep semantic parsing**.

An effective analogy is the relationship between high-level programming languages and low-level machine code. While natural language is comprehensible to humans, it is ambiguous and context dependent. A meaning representation, much like machine code, is structured, precise, and directly executable by a computer, making it comprehensible to machines but often incomprehensible to humans. The core challenge is to bridge this gap by developing techniques to interpret and encode the context and world knowledge inherent in human language.

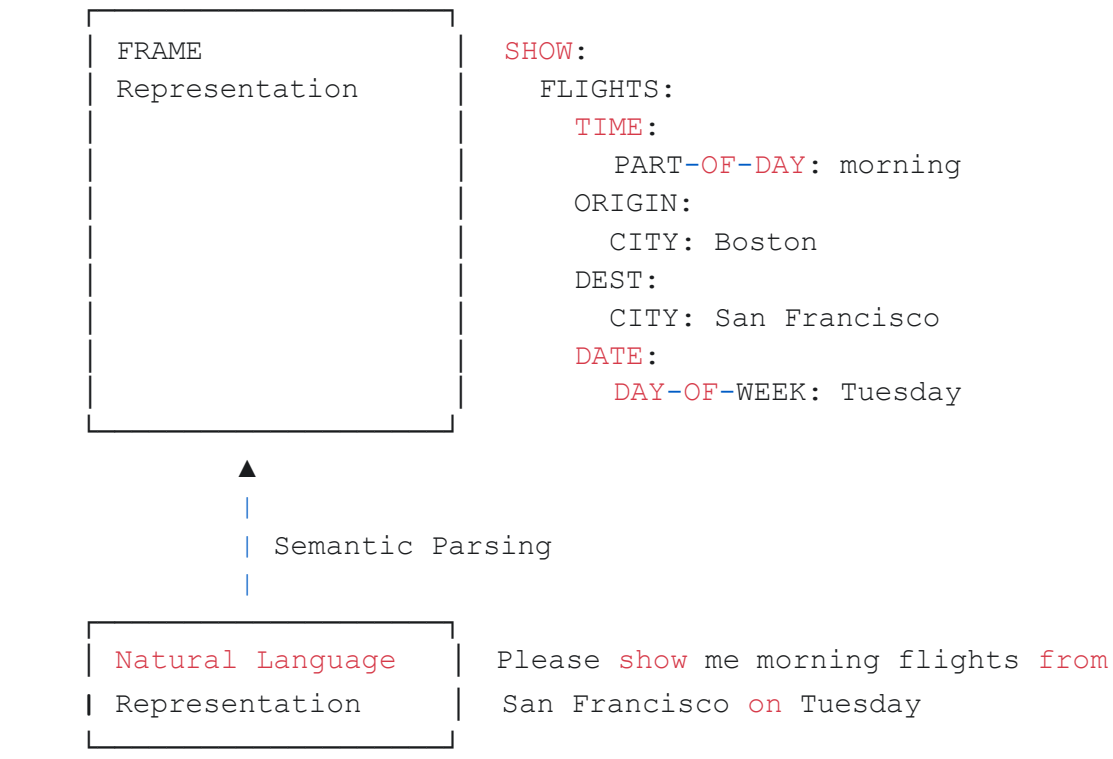
---

**Resources**

Several projects have created representations and resources that have promoted experimentation in this area. These resources typically provide a corpus of natural language inputs paired with their corresponding formal meaning representations, which are essential for training and evaluating systems.

- **ATIS (Air Travel Information System):** Considered one of the first major efforts, ATIS focused on transforming spoken user queries about flight information into a representation that could be compiled into a SQL query to interact with a flight database. The resource provides thousands of user utterances annotated with intermediate **hierarchical frame representations**. The diagram below illustrates this process:

**Diagram: Sample user query and its frame representation in the ATIS program**



The diagram shows how a natural language query is mapped to a structured frame. The frame captures the key semantic roles (like ORIGIN, DEST, TIME) and fills them with specific values (Boston, San Francisco, morning) extracted from the sentence.

This unambiguous representation can then be used by a system to perform an action, such as querying a database.

- **Communicator:** This program was a follow-on to ATIS that involved more complex **mixed-initiative dialogs**, where both the user and the machine could lead the conversation. The resource consists of thousands of collected dialogs related to travel planning.
- **GeoQuery:** This resource provides a natural language interface to a geographic database called Geobase. The corpus contains natural language questions paired with their formal representations as **Prolog queries**.
- For example, "What are the major cities in Kansas?" is mapped to

  **answer(C, (major(C), city(C), loc(C, S), equal(S, stateid(kansas)))).**

- **RoboCup: CLang:** In the domain of robotic soccer, this project uses a special formal language called **CLang** to encode advice from a team coach. The representation is expressed as **if-then ruies**. For instance, "If the ball is in our penalty area..." is mapped to ((bpos (penalty-area our)) ...).

---

**Systems**

Various systems have been developed to tackle the problem of mapping natural language to a meaning representation. These systems can be broadly categorized into rule-based and supervised approaches.

1. **Rule-Based Systems**

These systems use handcrafted **semantic grammar** to parse meaning units directly from the input. The philosophy is that underlying semantic information is less complex than a full syntactic explanation. This approach is robust against speech recognition errors and ungrammatical input.

- **Example System:** The **Phoenix** system used recursive transition networks (RTNs) and handcrafted grammar to extract a hierarchical frame structure for the ATIS and Communicator projects.

2. **Supervised Systems**

These systems use statistical models trained on hand-annotated data. They learn a mapping from natural language sentences to their formal meaning representations automatically.

**Example Systems:**

- **CHILL (Constructive Heuristics Induction for Language Learning):** Learns to map sentences into Prolog programs.
- **SCISSOR (Semantic Composition that Integrates Syntax and Semantics...):** Uses a statistical parser to create a **semantically augmented parse tree (SAPT)** to compositionally build the meaning representation.
- **KRISP (Kernel-based Robust Interpretation for Semantic Parsing):** Uses string kernels and Support Vector Machines (SVMs).
- **WASP (Word Alignment-based Semantic Parsing):** Treats semantic parsing as a machine translation problem, "translating" natural language into its meaning representation.

---

**Software**

While older rule-based systems are not widely available, several software programs for supervised semantic parsing systems are available for download and research, which are shown below:

- **WASP:** [http://www.cs.utexas.edu/~ml/wasp/]
- **KRISPER:** [http://www.cs.utexas.edu/~ml/krisp/]
- **CHILL:** [http://www.cs.utexas.edu/~ml/chill.html]

*****

**IMPORTANT QUESTIONS:**

1. ***Explain Predicate – Argument Structure.***

2. ***Explain Meaning Representation.***